

Hindi → Santali Offline AI App

Team Progress, Handover & Execution Guide

Owners: Aakansha — ML/Translation | Sharjil — UI/Speech/Performance

Current milestone: Working desktop offline proof-of-concept

The purpose of this document is to freeze the work completed so far and clearly divide the remaining work. Aakansha owns translation quality and model training. Sharjil owns the application, speech pipeline, offline integration and performance. They can work in parallel and integrate through a simple Hindi-text → Santali-text interface.

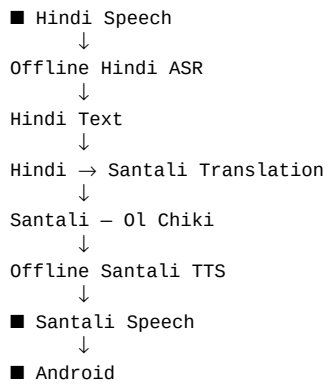
Area	Current status
Hindi → Santali	Working pretrained baseline
OI Chiki output	Working + script validation
Santali TTS	Working offline
Desktop prototype	Working
Fine-tuning	Not completed
Hindi speech / ASR	Not completed
Android	Not started
Mobile optimization	Not started

Important: The current translation model is a pretrained baseline, not a successfully fine-tuned Hindi → Santali model. Wrong translations are therefore a known baseline limitation and are the main ML task for Aakansha.

1. Final Goal

Build an offline Hindi → Santali voice translation application for Android. The user should speak Hindi, the phone should recognize it locally, translate it to Santali in Ol Chiki, and play Santali speech without internet.

Target pipeline



Requirement	Target
Translation	Correct and natural Hindi → Santali
Script	Ol Chiki
Internet	Not required during inference
Speech input	Offline Hindi ASR
Speech output	Offline Santali TTS
Response time	Target < 3 seconds end-to-end
Platform	Android

2. What Has Been Completed

Models integrated

Component	Selected model / implementation	Status
Translation	AI4Bharat IndicTrans2 Indic-Indic 1B	Integrated
Source	Hindi — hin_Deva	Configured
Target	Santali — sat_Olck	Configured
TTS	AI4Bharat Indic Parler-TTS	Integrated
TTS dependency	google/flan-t5-large	Cached locally

Offline work

- Models were downloaded and cached locally.
- Inference uses local model loading / local_files_only=True.
- The complete text → translation → speech pipeline was tested without external translation APIs.
- No OpenAI, Gemini, Google Translate or cloud translation API is used in the current inference path.

OI Chiki validation

- Implemented validation for the OI Chiki Unicode block U+1C50–U+1C7F.
- Foreign-script contamination can be detected.
- Script validity is not treated as proof of grammatical/semantic correctness.

Decoding investigation

- Repetition was investigated; repetition_penalty around 1.2 was useful for avoiding catastrophic loops.
- Naive hard vocabulary masking was not adopted because shared/subword tokenizer behavior can make it unsafe.

3. Dataset Prepared for Training

A clean Hindi → Santali parallel corpus was prepared for future fine-tuning.

Metric	Value
Raw candidate pairs	2,012
Valid canonical pairs	2,007
Rejected pairs	5
Training	1,605
Validation	200
Test	202
OI Chiki-valid target ratio	100% in prepared dataset
Duplicates	0 reported
Split seed	42

Processing already performed

- Unicode / whitespace normalization
- Hindi and OI Chiki validation
- Duplicate / near-duplicate analysis
- Length and alignment analysis
- Quality scoring
- Train/validation/test splitting
- Leakage checks
- Locked 100-example evaluation set

Important files

```
data/processed/train.jsonl
data/processed/validation.jsonl
data/processed/test.jsonl
data/evaluation/baseline_eval.jsonl

outputs/dataset_inventory.json
outputs/rejected_dataset.jsonl
outputs/deduplication_report.json
outputs/alignment_analysis.json
outputs/split_report.json
outputs/pretraining_baseline_results.json
outputs/phase3_dataset_report.md
```

4. Baseline & Working Prototype

Pre-training baseline

Metric	Result
BLEU-4	2.85
chrF++	30.41
OI Chiki validity	100% on fixed benchmark
Repetition	0% with current decoding setup
Foreign contamination	7% on fixed benchmark

These are baseline numbers, not final quality. The low BLEU and observed wrong translations are the reason the model must be improved.

Five-sentence offline proof-of-concept

[illegible]

Current prototype status

- 5/5 test sentences generated Santali audio.
- WAV output generated at 44.1 kHz.
- Translation and TTS loaded locally.
- Gradio desktop UI is running at <http://localhost:7860>.
- This is a desktop proof-of-concept, not yet the final Android app.

5. Training Status

Training has NOT been completed. A feasibility test was performed, but no successful fine-tuned checkpoint was produced.

Item	Result
CPU	Ryzen 5 5600H
GPU	No NVIDIA CUDA GPU on current development machine
Batch size in feasibility test	4
Forward + backward	~21.7 sec / step
Estimated 3 epochs	~7.25 hours
Peak RAM	~4.5 GB
Fine-tuned checkpoint	None

This is why Aakansha's GPU laptop is important. She should perform the actual GPU-based fine-tuning and evaluation.

Training rules

- Never overwrite the base model.
- Use 1,605 train / 200 validation / 202 test.
- Choose checkpoints using validation performance.
- Do not train on the test set.
- Compare base vs fine-tuned on automatic metrics and real qualitative examples.
- If quality is still poor, improve the data/error analysis rather than simply increasing epochs.

6. Aakansha — ML / Translation Owner

Main responsibility: Make Hindi → Santali translation correct, natural and reliable.

Tasks

- Run and document the current baseline.
- Inspect wrong translations and categorize errors.
- Fine-tune IndicTrans2 on the GPU laptop.
- Start with a controlled LoRA/PEFT experiment.
- Use validation data for checkpoint selection.
- Use the test set only for final evaluation.
- Improve/expand high-quality Hindi–Santali data if necessary.
- Get native Santali-speaker validation where possible.
- Deliver a reproducible best checkpoint and metrics report.

Definition of DONE

Aakansha is done when the fine-tuned model clearly improves over the baseline on unseen test data, does not create major script/repetition regressions, and has been checked qualitatively by a Santali speaker.

LLM PROMPT — COPY/PASTE INTO ANTIGRAVITY

You are working on the ML side of our Hindi → Santali offline translation project.

GOAL:

Improve Hindi → Santali translation quality for eventual offline Android use.

BASE MODEL:

ai4bharat/indictrans2-indic-indic-1B

Source: hin_Deva

Target: sat_Olck

DATA:

Train 1,605 | Validation 200 | Test 202

Locked evaluation: data/evaluation/baseline_eval.jsonl

BASELINE:

BLEU-4 2.85 | chrF++ 30.41

0l Chiki validity 100% on fixed benchmark

Repetition 0% with current decoding

Foreign contamination 7%

IMPORTANT:

The model has NOT been successfully fine-tuned yet.

Do not overwrite the base model.

Do not train on validation/test data.

Do not fabricate translations or metrics.

TASK:

1. Inspect the existing code and data.
2. Verify the GPU is being used.
3. Fine-tune using a controlled LoRA/PEFT experiment.
4. Record all hyperparameters and training time.
5. Save checkpoints separately.
6. Select the best checkpoint using validation performance.
7. Evaluate the selected checkpoint once on the locked test set.
8. Compare BASE vs FINE-TUNED using BLEU-4, chrF++, 0l Chiki validity, foreign contamination, repetition, empty outputs and latency.
9. Produce at least 30 qualitative examples.
10. Categorize translation errors.
11. If quality is still poor, investigate dataset quality and add high-quality Hindi-Santali data before simply increasing epochs.
12. Get native Santali-speaker validation if available.
13. Create outputs/aakansha_training_report.md.

Do NOT build Android, ASR or TTS.

Do NOT quantize or distill yet.

STOP after delivering the best validated model and report.

7. Sharjil — UI / Speech / Performance Owner

Main responsibility: Build the app, make speech work offline, and target <3 seconds end-to-end.

Tasks

- Use the current baseline model while Aakansha trains.
- Build a simple Android-first UI.
- Add offline Hindi ASR / microphone input.
- Integrate offline Santali TTS.
- Measure ASR, translation and TTS latency separately.
- Optimize the actual bottleneck.
- Target under 3 seconds from the end of Hindi speech to the beginning of Santali audio.
- Keep cloud APIs out of the final pipeline.
- Make it easy to replace the baseline translation model with Aakansha's final checkpoint.

Shared interfaces

```
translate_hindi_to_santali(hindi_text) -> santali_text
transcribe_hindi(audio_path) -> hindi_text
speak_santali(santali_text) -> audio_path
```

Definition of DONE

The Android prototype can record Hindi speech, recognize it locally, translate it, generate Santali speech locally, play it, and demonstrate offline operation. Latency must be measured rather than claimed.

LLM PROMPT — COPY/PASTE INTO ANTIGRAVITY

You are working on the UI, speech and performance side of our Hindi → Santali offline Android project.

GOAL:

Build an Android application that converts Hindi speech into Santali speech offline.

CURRENT WORKING BASELINE:

Hindi text → IndicTrans2 → Santali Ol Chiki → Indic Parler-TTS → WAV

YOUR JOB:

1. Do NOT retrain the translation model.
2. Use the current translation model as a temporary backend.
3. Build the Android UI.
4. Add genuinely offline Hindi ASR.
5. Integrate Santali TTS.
6. Measure ASR, translation and TTS latency separately.
7. Target <3 seconds end-to-end from end of Hindi speech to start of Santali audio.
8. Never claim <3 seconds unless measured.
9. Do not use Google Speech, Google Translate, OpenAI, Gemini, Azure, or any other cloud API.
10. Final inference must work without internet.

UI:

- Hindi → Santali title
- Offline indicator
- Microphone / hold-to-speak
- Recognized Hindi text
- Santali Ol Chiki output
- Play Santali audio
- Processing time
- Simple errors

PIPELINE:

Hindi speech → offline ASR → Hindi text → translation model
→ Santali text → offline TTS → audio

FIRST survey offline Hindi ASR options by accuracy, size, speed, Android compatibility, offline support and license.

THEN implement and benchmark the best practical option.

THEN build Android UI.

THEN benchmark model size, RAM, ASR latency, translation latency,

TTS latency and total latency.

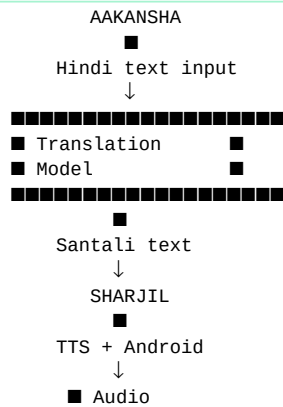
If total latency is above 3 seconds, identify and optimize the bottleneck.

Create outputs/sharjil_performance_report.md with actual measured numbers.

Do NOT train, fabricate metrics, or claim offline capability without testing.
STOP after the Android prototype and performance report work.

8. How They Work Together

Both people should work in parallel. Sharjil does not need to wait for Aakansha's final model.



Aakansha gives Sharjil

- Best translation checkpoint
- Tokenizer/configuration
- `translate_hindi_to_santali()` interface
- Translation metrics and limitations

Sharjil gives Aakansha

- Real-device latency
- ASR transcripts/failure cases
- TTS performance
- Android constraints
- Examples of slow or problematic end-to-end cases

Integration rule

Sharjil can build against the current baseline. When Aakansha produces a better checkpoint, replace the model behind the same translation interface. Do not rebuild the entire application.

9. Final Completion Checklist

Aakansha — Translation	Status
Baseline audit	■
GPU training	■
Best validation checkpoint	■
Locked test evaluation	■
Error analysis	■
Native Santali validation	■
Final model delivered	■
Sharjil — App / Speech / Performance	Status
Android project	■
Offline Hindi ASR	■
Microphone	■
Translation integration	■

Aakansha — Translation	Status
Offline Santali TTS	■
Audio playback	■
Latency benchmark	■
<3 sec optimization	■
Offline device test	■
Demo APK	■

Final acceptance test

1. Turn OFF internet.
2. Open Android app.
3. Speak Hindi.
4. Verify local Hindi transcription.
5. Verify correct Santali translation.
6. Verify Ol Chiki output.
7. Start Santali audio.
8. Measure total latency.
9. Confirm no cloud/API dependency.

Final success condition: A native Santali speaker confirms the translation is correct, the complete pipeline works offline on the target Android device, and measured latency meets the target or the remaining limitation is explicitly documented.